Step 1: Check If It Was Named app.py.txt

In your Command Prompt, type this command and press **Enter**:

```
DOS
dir *.py*
```

- **If it shows app.py.txt:** Run this rename command:

  ```
  DOS

  ren app.py.txt app.py
  ```

- **If it says File Not Found:** The file hasn't been created in this folder yet. Proceed to Step 2.

Step 2: Open app.py in Notepad

In the same Command Prompt, type:

```
DOS
notepad app.py
```

Press **Enter**. Windows will display a popup:

"Cannot find the app.py file. Do you want to create a new file?"

Click **Yes**.

Step 3: Paste and Save the Code

Paste this code into the open Notepad window:

```
Python
import streamlit as st
import pandas as pd
import sqlite3
import plotly.express as px
from datetime import datetime

# Page configuration
st.set_page_config(
    page_title="Local Food Wastage Management System",
    page_icon="🍲",
    layout="wide"
)

DB_NAME = "food_waste_management.db"

def get_connection():
    return sqlite3.connect(DB_NAME)

def run_query(query, params=()):
    with get_connection() as conn:
        return pd.read_sql_query(query, conn, params=params)

def execute_dml(query, params=()):
```

```
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(query, params)
        conn.commit()

# Initialize Database with the 1000-row CSVs
def init_db():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT name FROM sqlite_master WHERE type='table' AND
name='food_listings';")
    table_exists = cursor.fetchone()
    if not table_exists:
        try:
            p_df = pd.read_csv("providers_data.csv")
            r_df = pd.read_csv("receivers_data.csv")
            f_df = pd.read_csv("food_listings_data.csv")
            c_df = pd.read_csv("claims_data.csv")

            p_df.to_sql("providers", conn, if_exists="replace", index=False)
            r_df.to_sql("receivers", conn, if_exists="replace", index=False)
            f_df.to_sql("food_listings", conn, if_exists="replace", index=False)
            c_df.to_sql("claims", conn, if_exists="replace", index=False)
        except Exception as e:
            st.error(f"Error loading CSV files: {e}")
    conn.close()

init_db()

# Navigation Sidebar
st.sidebar.title("🍲 Food Waste System")
st.sidebar.markdown("**Waste Reduction & Social Good Platform**")
nav = st.sidebar.radio(
    "Navigation Menu",
    [
        "📊 Executive Dashboard",
        "🔍 Search & Claim Food",
        "➕ Manage Listings (CRUD)",
        "📑 15 SQL Business Insights",
        "👥 Community Directory"
    ]
)

# ---------------- PAGE 1: EXECUTIVE DASHBOARD ----------------
if nav == "📊 Executive Dashboard":
    st.title("🍲 Local Food Wastage Management System")
    st.markdown("Connecting surplus food providers with local NGOs and individuals in need.")

    total_food = run_query("SELECT SUM(Quantity) AS Total FROM food_listings;")['Total'].iloc[0] or 0
    total_listings = run_query("SELECT COUNT(*) AS Cnt FROM food_listings;")['Cnt'].iloc[0]
    total_claims = run_query("SELECT COUNT(*) AS Cnt FROM claims WHERE
Status='Completed';")['Cnt'].iloc[0]
    total_providers = run_query("SELECT COUNT(DISTINCT Provider_ID) AS Cnt FROM
providers;")['Cnt'].iloc[0]

    k1, k2, k3, k4 = st.columns(4)
    k1.metric("Available Food Units", f"{int(total_food):,} units")
    k2.metric("Total Food Listings", f"{total_listings:,}")
    k3.metric("Completed Claims", f"{total_claims:,}")
```

```
    k4.metric("Registered Providers", f"{total_providers:,}")

    st.markdown("---")

    col_l, col_r = st.columns(2)
    with col_l:
        st.subheader("Food Volume by Provider Type")
        df_prov_type = run_query("SELECT Provider_Type, SUM(Quantity) AS Total_Units FROM
food_listings GROUP BY Provider_Type ORDER BY Total_Units DESC;")
        fig_bar = px.bar(df_prov_type, x="Provider_Type", y="Total_Units", color="Provider_Type",
text="Total_Units", title="Food Units Contributed by Provider Type")
        st.plotly_chart(fig_bar, use_container_width=True)

    with col_r:
        st.subheader("Claims Status Distribution")
        df_status = run_query("SELECT Status, COUNT(*) AS Total FROM claims GROUP BY Status;")
        fig_pie = px.pie(df_status, names="Status", values="Total", hole=0.45, title="Claim Status
Distribution", color_discrete_sequence=px.colors.qualitative.Pastel)
        st.plotly_chart(fig_pie, use_container_width=True)

# ---------- PAGE 2: SEARCH & CLAIM FOOD ----------
elif nav == "🔍 Search & Claim Food":
    st.title("🔍 Search, Filter & Claim Available Food")
    st.markdown("Find available food donations based on location, food category, and meal type.")

    c1, c2, c3 = st.columns(3)
    cities = ["All"] + sorted(list(run_query("SELECT DISTINCT Location FROM
food_listings;")['Location']))
    food_types = ["All"] + sorted(list(run_query("SELECT DISTINCT Food_Type FROM
food_listings;")['Food_Type']))
    meal_types = ["All"] + sorted(list(run_query("SELECT DISTINCT Meal_Type FROM
food_listings;")['Meal_Type']))

    sel_city = c1.selectbox("Filter by City / Location", cities)
    sel_ftype = c2.selectbox("Filter by Food Type", food_types)
    sel_mtype = c3.selectbox("Filter by Meal Type", meal_types)

    base_sql = """
        SELECT f.Food_ID, f.Food_Name, f.Quantity, f.Expiry_Date, f.Food_Type, f.Meal_Type,
f.Location,
               p.Name AS Provider_Name, p.Contact AS Provider_Contact
        FROM food_listings f
        JOIN providers p ON f.Provider_ID = p.Provider_ID
        WHERE 1=1
    """
    params = []
    if sel_city != "All":
        base_sql += " AND f.Location = ?"
        params.append(sel_city)
    if sel_ftype != "All":
        base_sql += " AND f.Food_Type = ?"
        params.append(sel_ftype)
    if sel_mtype != "All":
        base_sql += " AND f.Meal_Type = ?"
        params.append(sel_mtype)

    df_filtered = run_query(base_sql, tuple(params))
    st.write(f"Showing **{len(df_filtered)}** available food listings:")
    st.dataframe(df_filtered, use_container_width=True)
```

```
    st.markdown("---")
    st.subheader("🛒 Claim Food Listing")
    with st.form("submit_claim_form"):
        col_f, col_r = st.columns(2)
        food_opts = run_query("SELECT Food_ID, Food_Name, Quantity, Location FROM food_listings
LIMIT 100;")
        rec_opts = run_query("SELECT Receiver_ID, Name, Type, City FROM receivers LIMIT 100;")

        sel_food_id = col_f.selectbox("Select Food Item", options=food_opts['Food_ID'],
format_func=lambda x: f"ID #{x} - {food_opts[food_opts['Food_ID']==x]['Food_Name'].iloc[0]}
({food_opts[food_opts['Food_ID']==x]['Location'].iloc[0]})")
        sel_rec_id = col_r.selectbox("Claiming Receiver Organization", options=rec_opts['Receiver_ID'],
format_func=lambda x: f"ID #{x} - {rec_opts[rec_opts['Receiver_ID']==x]['Name'].iloc[0]}
({rec_opts[rec_opts['Receiver_ID']==x]['Type'].iloc[0]})")

        btn_claim = st.form_submit_button("Submit Food Claim")
        if btn_claim:
            max_cid = run_query("SELECT MAX(Claim_ID) AS MaxID FROM claims;")['MaxID'].iloc[0] or
1000
            new_cid = int(max_cid) + 1
            now_str = datetime.now().strftime("%m/%d/%Y %H:%M")
            execute_dml("INSERT INTO claims (Claim_ID, Food_ID, Receiver_ID, Status, Timestamp)
VALUES (?, ?, ?, 'Pending', ?);",
                        (new_cid, sel_food_id, sel_rec_id, now_str))
            st.success(f"Claim #{new_cid} placed successfully with status 'Pending'!")

# ---------- PAGE 3: CRUD LISTINGS ----------
elif nav == "➕ Manage Listings (CRUD)":
    st.title("➕ Manage Food Listings (CRUD Operations)")
    st.markdown("Perform Create, Read, Update, and Delete operations on food listing records.")

    tab_create, tab_read, tab_update, tab_delete = st.tabs(["1. Create Listing", "2. View Listings", "3.
Update Listing", "4. Delete Listing"])

    with tab_create:
        st.subheader("List New Surplus Food Item (Create)")
        with st.form("form_create_food"):
            f_name = st.text_input("Food Item Name", "Fresh Rice and Dal Bowls")
            f_qty = st.number_input("Quantity Available (Units)", min_value=1, value=30)
            f_expiry = st.text_input("Expiry Date (e.g. 3/30/2025)", "3/30/2025")
            prov_df = run_query("SELECT Provider_ID, Name, Type, City FROM providers LIMIT 50;")
            prov_id = st.selectbox("Donor Provider", options=prov_df['Provider_ID'], format_func=lambda
x: f"#{x} - {prov_df[prov_df['Provider_ID']==x]['Name'].iloc[0]}
({prov_df[prov_df['Provider_ID']==x]['City'].iloc[0]})")
            ftype = st.selectbox("Food Type", ["Vegetarian", "Non-Vegetarian", "Vegan"])
            mtype = st.selectbox("Meal Type", ["Breakfast", "Lunch", "Dinner", "Snacks"])

            btn_add = st.form_submit_button("Submit New Listing")
            if btn_add:
                max_fid = run_query("SELECT MAX(Food_ID) AS MaxID FROM
food_listings;")['MaxID'].iloc[0] or 1000
                new_fid = int(max_fid) + 1
                prov_row = prov_df[prov_df['Provider_ID'] == prov_id].iloc[0]
                execute_dml("""
                    INSERT INTO food_listings (Food_ID, Food_Name, Quantity, Expiry_Date, Provider_ID,
Provider_Type, Location, Food_Type, Meal_Type)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?);
                """, (new_fid, f_name, f_qty, f_expiry, prov_id, prov_row['Type'], prov_row['City'], ftype,
mtype))
                st.success(f"Listing #{new_fid} '{f_name}' created successfully!")
```

```
    with tab_read:
        st.subheader("Current Active Listings (Read)")
        df_all = run_query("SELECT * FROM food_listings ORDER BY Food_ID DESC LIMIT 25;")
        st.dataframe(df_all, use_container_width=True)

    with tab_update:
        st.subheader("Modify Existing Food Listing (Update)")
        listings_edit = run_query("SELECT Food_ID, Food_Name, Quantity, Expiry_Date FROM
food_listings ORDER BY Food_ID DESC LIMIT 100;")
        edit_fid = st.selectbox("Select Food ID to Edit", options=listings_edit['Food_ID'],
format_func=lambda x: f"#{x} - {listings_edit[listings_edit['Food_ID']==x]['Food_Name'].iloc[0]}")
        selected_curr = listings_edit[listings_edit['Food_ID'] == edit_fid].iloc[0]

        with st.form("form_update_food"):
            up_qty = st.number_input("New Quantity", value=int(selected_curr['Quantity']), min_value=0)
            up_exp = st.text_input("New Expiry Date", value=str(selected_curr['Expiry_Date']))
            btn_update = st.form_submit_button("Update Listing")
            if btn_update:
                execute_dml("UPDATE food_listings SET Quantity = ?, Expiry_Date = ? WHERE Food_ID
= ?;", (up_qty, up_exp, edit_fid))
                st.success(f"Food Listing #{edit_fid} updated successfully!")

    with tab_delete:
        st.subheader("Remove Food Listing (Delete)")
        del_list = run_query("SELECT Food_ID, Food_Name, Location FROM food_listings ORDER BY
Food_ID DESC LIMIT 100;")
        del_fid = st.selectbox("Select Food ID to Delete", options=del_list['Food_ID'],
format_func=lambda x: f"#{x} - {del_list[del_list['Food_ID']==x]['Food_Name'].iloc[0]}
({del_list[del_list['Food_ID']==x]['Location'].iloc[0]})")
        if st.button("Delete Listing Permanently", type="primary"):
            execute_dml("DELETE FROM food_listings WHERE Food_ID = ?;", (del_fid,))
            st.warning(f"Food Listing #{del_fid} removed from the database.")

# ---------- PAGE 4: 15 SQL BUSINESS QUERIES ----------
elif nav == "📑 15 SQL Business Insights":
    st.title("📑 15 Mandatory SQL Business Queries")
    st.markdown("Real-time execution of the core analytical queries requested in the project brief.")

    queries_dict = {
        "Q1: How many food providers and receivers are there in each city?": """
SELECT p.City, COUNT(DISTINCT p.Provider_ID) AS Total_Providers, COUNT(DISTINCT
r.Receiver_ID) AS Total_Receivers
FROM providers p
LEFT JOIN receivers r ON p.City = r.City
GROUP BY p.City
ORDER BY Total_Providers DESC
LIMIT 15;
        """,
        "Q2: Which type of food provider contributes the most food?": """
SELECT Provider_Type, SUM(Quantity) AS Total_Food_Units, COUNT(Food_ID) AS Total_Listings
FROM food_listings
GROUP BY Provider_Type
ORDER BY Total_Food_Units DESC;
        """,
        "Q3: What is the contact information of food providers in a specific city?": """
SELECT Name, Type, Address, City, Contact
FROM providers
ORDER BY City, Name
LIMIT 15;
```

```
    """,
    "Q4: Which receivers have claimed the most food?": """
SELECT r.Name AS Receiver_Name, r.Type, COUNT(c.Claim_ID) AS Total_Claims, SUM(f.Quantity)
AS Total_Quantity_Claimed
FROM claims c
JOIN receivers r ON c.Receiver_ID = r.Receiver_ID
JOIN food_listings f ON c.Food_ID = f.Food_ID
GROUP BY r.Name, r.Type
ORDER BY Total_Quantity_Claimed DESC
LIMIT 15;
    """,
    "Q5: What is the total quantity of food available from all providers?": """
SELECT SUM(Quantity) AS Total_Available_Food_Units, COUNT(Food_ID) AS Total_Listings
FROM food_listings;
    """,
    "Q6: Which city has the highest number of food listings?": """
SELECT Location AS City, COUNT(Food_ID) AS Total_Listings, SUM(Quantity) AS
Total_Food_Quantity
FROM food_listings
GROUP BY Location
ORDER BY Total_Listings DESC
LIMIT 15;
    """,
    "Q7: What are the most commonly available food types?": """
SELECT Food_Type, COUNT(Food_ID) AS Listing_Count, SUM(Quantity) AS Total_Quantity
FROM food_listings
GROUP BY Food_Type
ORDER BY Total_Quantity DESC;
    """,
    "Q8: How many food claims have been made for each food item?": """
SELECT f.Food_Name, COUNT(c.Claim_ID) AS Total_Claims
FROM food_listings f
LEFT JOIN claims c ON f.Food_ID = c.Food_ID
GROUP BY f.Food_Name
ORDER BY Total_Claims DESC;
    """,
    "Q9: Which provider has had the highest number of successful food claims?": """
SELECT p.Name AS Provider_Name, p.Type, COUNT(c.Claim_ID) AS Successful_Claims
FROM claims c
JOIN food_listings f ON c.Food_ID = f.Food_ID
JOIN providers p ON f.Provider_ID = p.Provider_ID
WHERE c.Status = 'Completed'
GROUP BY p.Name, p.Type
ORDER BY Successful_Claims DESC
LIMIT 15;
    """,
    "Q10: What percentage of food claims are completed vs. pending vs. canceled?": """
SELECT Status, COUNT(Claim_ID) AS Total_Claims,
    ROUND(COUNT(Claim_ID) * 100.0 / (SELECT COUNT(*) FROM claims), 2) AS Percentage
FROM claims
GROUP BY Status;
    """,
    "Q11: What is the average quantity of food claimed per receiver?": """
SELECT r.Name AS Receiver_Name, ROUND(AVG(f.Quantity), 2) AS Avg_Claim_Quantity
FROM claims c
JOIN receivers r ON c.Receiver_ID = r.Receiver_ID
JOIN food_listings f ON c.Food_ID = f.Food_ID
WHERE c.Status = 'Completed'
GROUP BY r.Name
ORDER BY Avg_Claim_Quantity DESC
```

```
LIMIT 15;
        """,
        "Q12: Which meal type is claimed the most?": """
SELECT f.Meal_Type, COUNT(c.Claim_ID) AS Total_Claims
FROM claims c
JOIN food_listings f ON c.Food_ID = f.Food_ID
GROUP BY f.Meal_Type
ORDER BY Total_Claims DESC;
        """,
        "Q13: What is the total quantity of food donated by each provider?": """
SELECT p.Name AS Provider_Name, p.City, SUM(f.Quantity) AS Total_Quantity_Donated
FROM food_listings f
JOIN providers p ON f.Provider_ID = p.Provider_ID
GROUP BY p.Name, p.City
ORDER BY Total_Quantity_Donated DESC
LIMIT 15;
        """,
        "Q14: Which food listings are near-expiry or require urgent distribution?": """
SELECT Food_Name, Expiry_Date, Location, Quantity
FROM food_listings
ORDER BY Expiry_Date ASC
LIMIT 15;
        """,
        "Q15: What is the city-wise food distribution efficiency (Listings vs Claims)?": """
SELECT p.City, COUNT(DISTINCT f.Food_ID) AS Total_Food_Listings, COUNT(DISTINCT
c.Claim_ID) AS Total_Claims_Processed
FROM food_listings f
JOIN providers p ON f.Provider_ID = p.Provider_ID
LEFT JOIN claims c ON f.Food_ID = c.Food_ID
GROUP BY p.City
ORDER BY Total_Claims_Processed DESC
LIMIT 15;
        """
    }

    choice = st.selectbox("Select SQL Query to Execute", list(queries_dict.keys()))
    sql_stmt = queries_dict[choice]

    st.markdown("**Executed SQL Query:**")
    st.code(sql_stmt, language="sql")

    df_output = run_query(sql_stmt)
    st.markdown("**Live Result:**")
    st.dataframe(df_output, use_container_width=True)

# ---------- PAGE 5: DIRECTORY ----------
elif nav == "👥 Community Directory":
    st.title("👥 Community Directory")
    st.markdown("Direct contact details of all registered donors and verified receiver organizations.")

    col1, col2 = st.columns(2)
    with col1:
        st.subheader("Food Providers")
        df_p = run_query("SELECT Provider_ID, Name, Type, City, Contact, Address FROM providers
LIMIT 100;")
        st.dataframe(df_p, use_container_width=True)
    with col2:
        st.subheader("Verified Receivers")
        df_r = run_query("SELECT Receiver_ID, Name, Type, City, Contact FROM receivers LIMIT
100;")
```

```
    st.dataframe(df_r, use_container_width=True)
```

Press **Ctrl + S** in Notepad to save, then close Notepad.

Step 4: Run the Application

In your Command Prompt window, run:

DOS

```
py -m streamlit run app.py
```

This prompt for streamlit application

py -m streamlit run app.py